

Contents

Real-Time Real Estate Backend Service	3
Table of Contents	3
Overview	3
Technology Stack	3
?Features	4
Property Services	4
?Geolocation Services	4
Investment Analysis	4
?Architecture	4
Quick Start	5
Prerequisites	5
Installation	5
Using Batch Scripts (Windows)	6
API Documentation	6
Base URL	6
Endpoints Overview	6
Detailed API Reference	6
Configuration	8
Application Configuration (<code>application.yml</code>)	8
Environment Variables	8
Project Structure	8
Related Documentation	9
Troubleshooting	9
Common Issues	9
License	10
Contributors	10
Support	10
 User Guide - Real Estate Backend Service	 10
Table of Contents	10
1. Introduction	11
What is this application?	11
Who should use this guide?	11
2. System Requirements	11
Minimum Requirements	11
Required Accounts	11
3. Installation Guide	11
Step 1: Install Java 17	11
Step 2: Install Maven	12
Step 3: Get RapidAPI Key	13
Step 4: Download the Project	13
4. Configuration	13
Setting Up Environment Variables	13
Configuration File	13
Changing the Server Port	14
5. Starting the Server	14

Method 1: Using Maven (Recommended)	14
Method 2: Using Batch Scripts (Windows)	14
Method 3: Using JAR File	14
Verifying the Server is Running	14
6. Using the API	15
Property Search	15
Get Property Details	15
Investment Cashflow Analysis	15
Geolocation Search	15
7. Troubleshooting	16
Issue: “Port 8080 already in use”	16
Issue: “RAPIDAPI_KEY not found”	16
Issue: “Connection refused” errors	16
Issue: “401 Unauthorized” from API	16
Issue: “CORS error” in browser	16
8. FAQ	17
Q: How do I change the server port?	17
Q: How many API calls can I make?	17
Q: Can I run this on a remote server?	17
Q: How do I update the application?	17
Q: Is my API key secure?	17
Q: How do I view server logs?	17
Getting Help	17
Document History	17
Developer Guide - Real Estate Backend Service	18
Table of Contents	18
1. Architecture Overview	18
System Architecture	18
Design Patterns Used	19
2. Development Environment Setup	19
Prerequisites	19
IDE Setup	19
Environment Configuration	20
Running in Development Mode	20
3. Code Structure	20
Package Organization	20
Naming Conventions	21
4. API Reference	21
REST Endpoints Summary	21
5. Core Components	23
5.1 Application Entry Point	23
5.2 Zillow Service	23
5.3 Cashflow Service	24
6. Data Models	24
6.1 CashflowRequest (Input DTO)	24
6.2 CashflowResponse (Output DTO)	26
6.3 EnrichedProperty	28

7. Business Logic	29
7.1 Cashflow Calculation Algorithm	29
7.2 IRR Calculation (Newton-Raphson)	30
8. Testing	31
Running Tests	31
Test Structure	31
Writing Tests	31
9. Code Style Guidelines	32
Java Style	32
Formatting Rules	33
10. Contributing	33
Git Workflow	33
Commit Message Format	33
11. Generating Documentation	33
Using Doxygen (Optional)	33
Using Pandoc for PDF Export	34
Document History	34

Real-Time Real Estate Backend Service

A comprehensive Spring Boot backend service for real estate investment analysis. This service provides RESTful APIs for property search, geolocation services, and investment cashflow analysis.

Table of Contents

- Overview
 - Features
 - Architecture
 - Quick Start
 - API Documentation
 - Configuration
 - Project Structure
 - Related Documentation
-

Overview

This backend service is part of the **Real-Time Real Estate Investment Analysis Web Application**. It serves as the central API gateway that:

1. **Integrates with Zillow API** - Fetches property data, estimates, and market information
2. **Provides Geolocation Services** - Uses Google API for geocoding and place search
3. **Performs Investment Analysis** - Calculates cashflow projections and investment metrics

Technology Stack

Using Batch Scripts (Windows)

```
# Compile the project
compile.bat

# Start the backend server
start-backend.bat
```

API Documentation

Base URL

`http://localhost:8080`

Endpoints Overview

Method	Endpoint	Description
GET	<code>/api/properties/search</code>	Search properties by location
GET	<code>/api/properties/{zpid}</code>	Get property details
POST	<code>/api/analysis/cashflow</code>	Analyze investment cashflow
GET	<code>/api/geo/text</code>	Text-based place search
GET	<code>/api/geo/circle</code>	Radius-based search
POST	<code>/api/geo/raw</code>	Raw geo query

Detailed API Reference

1. Property Search

GET `/api/properties/search?location={location}&status={status}&page={page}`

Parameters:

Name	Type	Required	Description
location	string	Yes	City, state, or ZIP code
status	string	No	for_sale, for_rent, sold
page	integer	No	Page number (default: 1)

Example:

```
curl "http://localhost:8080/api/properties/search?location=Santa%20Monica,%20CA&status=for_sale"
```

2. Property Details

GET `/api/properties/{zpid}`

Example:

```
curl "http://localhost:8080/api/properties/20479916"
```

Response:

```
{
  "zpid": "20479916",
  "address": "123 Main St, Santa Monica, CA 90401",
}
```

```
"price": 1500000,  
"bedrooms": 3,  
"bathrooms": 2,  
"livingArea": 1800,  
"yearBuilt": 1985,  
"zestimate": 1450000,  
"rentEstimate": 5500,  
"annualTax": 15000,  
"hoaMonthly": 350  
}
```

3. Cashflow Analysis

POST /api/analysis/cashflow

Content-Type: application/json

Request Body:

```
{  
  "offerPrice": 500000,  
  "fmv": 550000,  
  "grossRentsAnnual": 48000,  
  "numberOfUnits": 4,  
  "vacancyRate": 0.05,  
  "managementRate": 0.10,  
  "propertyTaxes": 6000,  
  "insurance": 2400,  
  "firstPrincipal": 400000,  
  "firstRateAnnual": 0.07,  
  "firstAmortYears": 30,  
  "holdYears": 10  
}
```

Response:

```
{  
  "summary": {  
    "rpp": 500000,  
    "cashToClose": 100000,  
    "noiY1": 35000,  
    "capRatePPY1": 0.07,  
    "dscrY1": 1.15,  
    "cashOnCashY1": 0.085,  
    "irr": 0.12,  
    "equityMultiple": 2.1  
  },  
  "projection": [  
    { "year": 1, "noi": 35000, "cashFlowBeforeTax": 8500 },  
    { "year": 2, "noi": 36050, "cashFlowBeforeTax": 9100 }  
  ]  
}
```

```
}
```

4. Geo Text Search

```
GET /api/geo/text?text={text}&city={city}&state={state}
```

Example:

```
curl "http://localhost:8080/api/geo/text?text=white%20house&city=washington&state=DC"
```

5. Geo Circle Search

```
GET /api/geo/circle?lat={lat}&lon={lon}&radius={radius}&text={text}
```

Example:

```
curl "http://localhost:8080/api/geo/circle?lat=34.0195&lon=-118.4912&radius=5000&text=restaurant"
```

Configuration

Application Configuration (application.yml)

```
server:
  port: 8080

zillow:
  rapidapi:
    host: zillow-com1.p.rapidapi.com
    key: ${RAPIDAPI_KEY:your-default-key}

googleapi31:
  rapidapi:
    host: google-api31.p.rapidapi.com
    key: ${RAPIDAPI_KEY:your-default-key}
  endpoint: https://google-api31.p.rapidapi.com/map
```

Environment Variables

Variable	Description	Required
RAPIDAPI_KEY	RapidAPI authentication key	Yes
SERVER_PORT	Server port (default: 8080)	No

Project Structure

```
googlemapv2/
├── src/
└── ?    └── main/
```



```

? ?    € java/com/example/
? ? ?   € map/
? ? ? ?   € Application.java          # Main entry point
? ? ? ?   € config/
? ? ? ? ?   € CorsConfig.java         # CORS configuration
? ? ? ?   € geo/
? ? ? ?   € GeoController.java        # Geo REST endpoints
? ? ? ? ?   € GoogleApi31Service.java
? ? ? ?   € realestate/
? ? ? ?   € controller/
? ? ? ? ?   € PropertyController.java
? ? ? ?   € model/
? ? ? ? ?   € EnrichedProperty.java
? ? ? ? ?   € SearchCard.java
? ? ? ?   € service/
? ? ? ? ?   € ZillowService.java
? ? ? ?   € analysis/
? ? ? ?   € controller/
? ? ? ? ?   € CashflowController.java
? ? ? ?   € dto/
? ? ? ? ?   € CashflowRequest.java
? ? ? ? ?   € CashflowResponse.java
? ? ? ?   € service/
? ? ? ?   € CashflowService.java
? ?    € resources/
? ?    € application.yml
?    € test/
?    € java/com/example/
    € pom.xml
    € README.md
    € USER_GUIDE.md
    € DEVELOPER_GUIDE.md

```

Related Documentation

Document	Description
USER_GUIDE.md	End-user instructions for setup and usage
DEVELOPER_GUIDE.md	Technical documentation for developers
API Reference	HTTP request examples

Troubleshooting

Common Issues

1. Port 8080 already in use

```
# Windows
netstat -ano | findstr :8080
taskkill /PID <PID> /F
```

```
# Linux/macOS
lsof -i :8080
kill -9 <PID>
```

2. API Key errors - Verify RAPIDAPI_KEY environment variable is set - Check RapidAPI subscription status - Ensure API key has access to required endpoints

3. CORS errors - Backend includes CORS configuration for localhost:3000 - Add additional origins in CorsConfig.java if needed

License

This project is licensed under the MIT License - see the LICENSE file for details.

Contributors

- **Praneeth Vadrevu** - Project Owner
- CS 682 - Fall 2025

Support

For questions or issues, please create an issue on the GitHub repository.

User Guide - Real Estate Backend Service

Version: 1.0.0

Last Updated: December 2025

Audience: End Users, System Administrators

Table of Contents

1. Introduction
2. System Requirements
3. Installation Guide
4. Configuration
5. Starting the Server
6. Using the API
7. Troubleshooting
8. FAQ

1. Introduction

What is this application?

The **Real Estate Backend Service** is a RESTful API server that powers the Real-Time Real Estate Investment Analysis Web Application. It provides:

- **Property Search** - Search real estate listings from Zillow
- **Geolocation** - Convert addresses to coordinates and search nearby places
- **Investment Analysis** - Calculate ROI, cashflow, and other investment metrics

Who should use this guide?

- **System Administrators** setting up the backend server
 - **Developers** integrating with the API
 - **End Users** running the application locally
-

2. System Requirements

Minimum Requirements

Component	Requirement
Operating System	Windows 10/11, macOS 10.15+, Linux (Ubuntu 20.04+)
Java	JDK 17 or higher
Memory	4 GB RAM minimum
Disk Space	500 MB free space
Network	Internet connection (for API calls)

Required Accounts

Service	Purpose	Link
RapidAPI	API access for Zillow & Google services	Sign Up

3. Installation Guide

Step 1: Install Java 17

Windows

1. Download Java 17 from Adoptium
2. Run the installer

3. Verify installation:

```
java -version
```

Expected output: openjdk version "17.x.x"

macOS

```
# Using Homebrew
```

```
brew install openjdk@17
```

```
# Add to PATH
```

```
echo 'export PATH="/opt/homebrew/opt/openjdk@17/bin:$PATH"' >> ~/.zshrc
```

```
source ~/.zshrc
```

```
# Verify
```

```
java -version
```

Linux (Ubuntu/Debian)

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk
```

```
# Verify
```

```
java -version
```

Step 2: Install Maven

Windows

1. Download Maven from Apache Maven
2. Extract to C:\Program Files\Apache\maven
3. Add to System PATH:
 - Open System Properties → Environment Variables
 - Add C:\Program Files\Apache\maven\bin to PATH
4. Verify:

```
mvn -version
```

macOS/Linux

```
# macOS
```

```
brew install maven
```

```
# Linux
```

```
sudo apt install maven
```

```
# Verify
```

```
mvn -version
```

Step 3: Get RapidAPI Key

1. Go to RapidAPI
2. Create an account or sign in
3. Subscribe to these APIs:
 - Zillow API
 - Google API 31
4. Copy your API key from the dashboard

Step 4: Download the Project

Clone the repository

```
git clone https://github.com/praneeth-vadrevu/Real-Time-Real-Estate-Investment-Analysis--Web-Ap
```

Navigate to backend directory

```
cd Real-Time-Real-Estate-Investment-Analysis--Web-App/googlemapv2
```

4. Configuration

Setting Up Environment Variables

Windows (PowerShell)

Temporary (current session only)

```
$env:RAPIDAPI_KEY = "your-api-key-here"
```

Permanent

```
[Environment]::SetEnvironmentVariable("RAPIDAPI_KEY", "your-api-key-here", "User")
```

Windows (Command Prompt)

```
set RAPIDAPI_KEY=your-api-key-here
```

macOS/Linux

Temporary

```
export RAPIDAPI_KEY="your-api-key-here"
```

Permanent (add to ~/.bashrc or ~/.zshrc)

```
echo 'export RAPIDAPI_KEY="your-api-key-here"' >> ~/.bashrc
```

```
source ~/.bashrc
```

Configuration File

The main configuration file is located at `src/main/resources/application.yml`:

```
server:
```

```
  port: 8080                                # Server port (change if needed)
```

```
zillow:
```

```
rapidapi:
  host: zillow-com1.p.rapidapi.com
  key: ${RAPIDAPI_KEY}      # Uses environment variable

googleapi31:
  rapidapi:
    host: google-api31.p.rapidapi.com
    key: ${RAPIDAPI_KEY}
  endpoint: https://google-api31.p.rapidapi.com/map
```

Changing the Server Port

If port 8080 is in use, modify application.yml:

```
server:
  port: 8081  # or any available port
```

5. Starting the Server

Method 1: Using Maven (Recommended)

```
# Navigate to the backend directory
cd googlemapv2

# Build and run
mvn spring-boot:run
```

Method 2: Using Batch Scripts (Windows)

```
# First time: compile the project
compile.bat

# Start the server
start-backend.bat
```

Method 3: Using JAR File

```
# Build the JAR
mvn clean package

# Run the JAR
java -jar target/googlemapv2-1.0.0.jar
```

Verifying the Server is Running

1. Check the console output:

Started Application in X.XXX seconds (process running for X.XXX)

2. Test the API:

```
curl http://localhost:8080/api/geo/text?text=test
```

3. **Open in browser:** Navigate to <http://localhost:8080/api/geo/text?text=test>

6. Using the API

Property Search

Search for properties in a specific location:

```
# Search for homes for sale in Santa Monica
```

```
curl "http://localhost:8080/api/properties/search?location=Santa%20Monica,%20CA&status=for_sale"
```

Parameters: - **location** (required): City, state, or ZIP code - **status** (optional): **for_sale**, **for_rent**, or **sold** - **page** (optional): Page number for pagination

Get Property Details

Get detailed information about a specific property:

```
# Get property by Zillow Property ID (ZPID)
```

```
curl "http://localhost:8080/api/properties/20479916"
```

Investment Cashflow Analysis

Analyze the investment potential of a property:

```
curl -X POST "http://localhost:8080/api/analysis/cashflow" \
-H "Content-Type: application/json" \
-d '{
  "offerPrice": 500000,
  "fmv": 550000,
  "grossRentsAnnual": 48000,
  "numberOfUnits": 4,
  "vacancyRate": 0.05,
  "managementRate": 0.10,
  "propertyTaxes": 6000,
  "insurance": 2400,
  "firstPrincipal": 400000,
  "firstRateAnnual": 0.07,
  "firstAmortYears": 30,
  "holdYears": 10
}'
```

Geolocation Search

Search for places by text:

```
curl "http://localhost:8080/api/geo/text?text=coffee&city=Boston&state=MA"
```

Search within a radius:

```
curl "http://localhost:8080/api/geo/circle?lat=34.0195&lon=-118.4912&radius=5000&text=restaurant"
```

7. Troubleshooting

Issue: “Port 8080 already in use”

Solution:

Windows:

```
# Find the process using port 8080
```

```
netstat -ano | findstr :8080
```

```
# Kill the process (replace <PID> with actual PID)
```

```
taskkill /PID <PID> /F
```

macOS/Linux:

```
# Find and kill the process
```

```
lsof -i :8080
```

```
kill -9 <PID>
```

Or change the port in `application.yml`.

Issue: “RAPIDAPI_KEY not found”

Solution: 1. Verify the environment variable is set: powershell echo \$env:RAPIDAPI_KEY
Windows PowerShell echo \$RAPIDAPI_KEY # Linux/macOS

2. If empty, set it again (see Configuration section)

3. Restart your terminal/IDE after setting

Issue: “Connection refused” errors

Solution: 1. Verify the server is running 2. Check if a firewall is blocking the connection 3. Ensure you’re using the correct port

Issue: “401 Unauthorized” from API

Solution: 1. Check if your RapidAPI key is valid 2. Verify you’re subscribed to the required APIs
3. Check if you’ve exceeded API rate limits

Issue: “CORS error” in browser

Solution: The backend allows CORS from `localhost:3000` by default. If your frontend runs on a different port, update `CorsConfig.java`.

8. FAQ

Q: How do I change the server port?

A: Edit `src/main/resources/application.yml` and change `server.port` to your desired port.

Q: How many API calls can I make?

A: This depends on your RapidAPI subscription plan. Check your RapidAPI dashboard for limits.

Q: Can I run this on a remote server?

A: Yes! Deploy the JAR file to any server with Java 17 and set the environment variables accordingly.

Q: How do I update the application?

A:

```
git pull origin main
mvn clean install
mvn spring-boot:run
```

Q: Is my API key secure?

A: The API key is stored as an environment variable, not in the code. Never commit API keys to version control.

Q: How do I view server logs?

A: Logs are printed to the console. For file logging, add this to `application.yml`:

```
logging:
  file:
    name: backend.log
```

Getting Help

- **GitHub Issues:** Report a bug
 - **Documentation:** Check the README.md for quick reference
-

Document History

Version	Date	Author	Changes
1.0.0	Dec 2025	Team	Initial release

Developer Guide - Real Estate Backend Service

Version: 1.0.0

Last Updated: December 2025

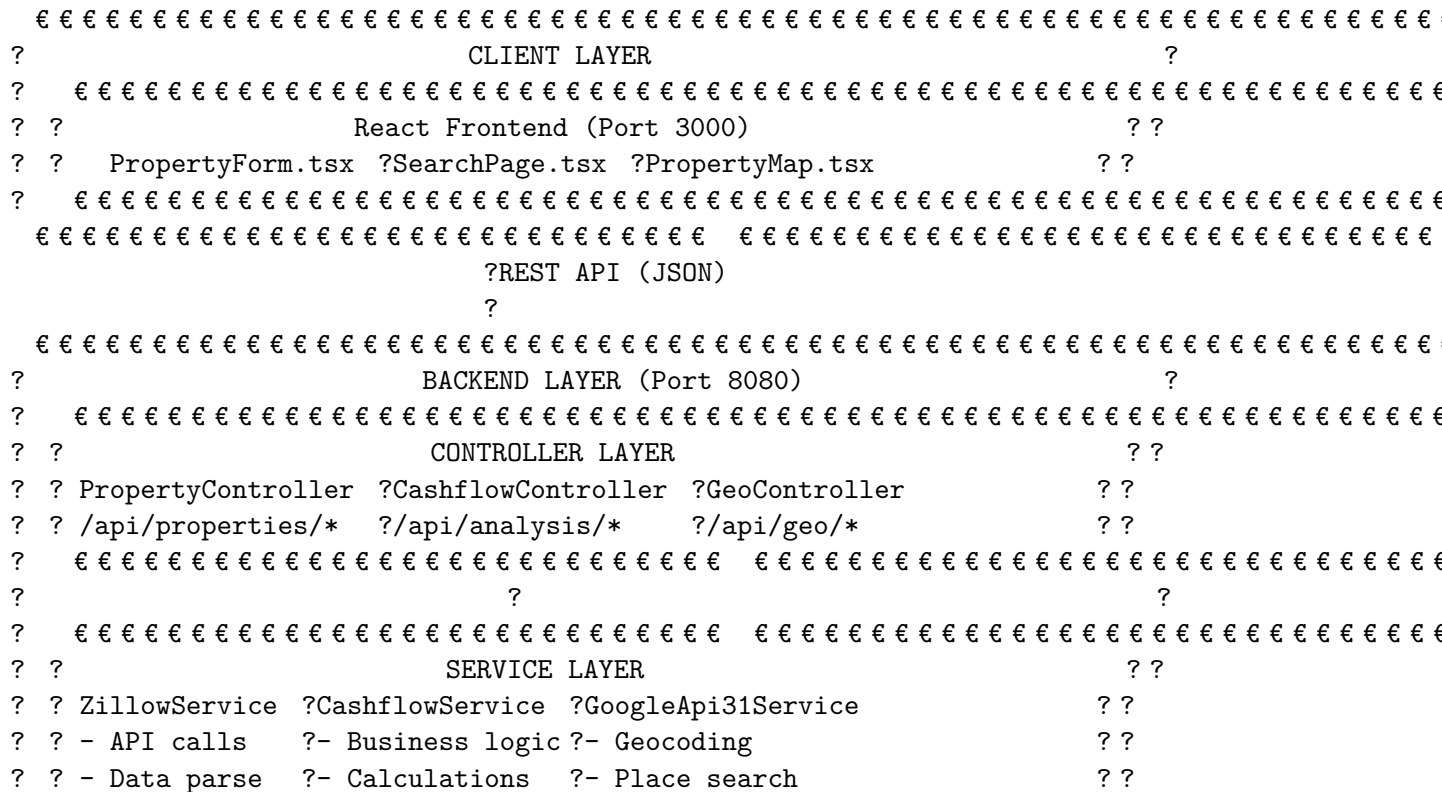
Audience: Developers, Contributors

Table of Contents

1. Architecture Overview
 2. Development Environment Setup
 3. Code Structure
 4. API Reference
 5. Core Components
 6. Data Models
 7. Business Logic
 8. Testing
 9. Code Style Guidelines
 10. Contributing
 11. Generating Documentation
-

1. Architecture Overview

System Architecture



4. Mark `src/main/java` as Sources Root
5. Mark `src/test/java` as Test Sources Root

VS Code

1. Install extensions:
 - Extension Pack for Java
 - Spring Boot Extension Pack
2. Open the `googlemapv2` folder
3. Wait for Java project recognition

Environment Configuration

Create a `.env` file (do NOT commit):

```
# .env (add to .gitignore)
RAPIDAPI_KEY=your-api-key-here
```

Running in Development Mode

```
# Hot reload enabled
mvn spring-boot:run -Dspring-boot.run.jvmArguments="-Dspring.devtools.restart.enabled=true"

# Debug mode (port 5005)
mvn spring-boot:run -Dspring-boot.run.jvmArguments="-agentlib:jdwp=transport=dt_socket,server=
```

3. Code Structure

Package Organization

```
src/main/java/com/example/
  € map/                                # Application core
  ?   € Application.java                # Spring Boot main class
  ?   € config/
  ?   ?   € CorsConfig.java            # CORS configuration
  ?   € geo/
  ?       € GeoController.java          # Geo REST endpoints
  ?       € GoogleApi31Service.java     # Google API integration
  ?
  € realestate/                          # Real estate domain
  ?   € controller/
  ?   ?   € PropertyController.java     # Property REST endpoints
  ?   € model/
  ?   ?   € EnrichedProperty.java       # Full property data model
  ?   ?   € SearchCard.java            # Search result model
  ?   € service/
  ?       € ZillowService.java          # Zillow API integration
  ?
  € analysis/                           # Investment analysis
```

```

    € controller/
?    € CashflowController.java # Analysis REST endpoints
    € dto/
?    € CashflowRequest.java    # Input DTO
?    € CashflowResponse.java   # Output DTO
    € service/
        € CashflowService.java # Cashflow calculations

```

Naming Conventions

Type	Convention	Example
Classes	PascalCase	CashflowService
Methods	camelCase	analyzeProperty()
Constants	UPPER_SNAKE	MAX_RETRY_COUNT
Variables	camelCase	propertyPrice
Packages	lowercase	com.example.analysis

4. API Reference

REST Endpoints Summary

Property Controller (/api/properties)

```

/**
 * Search properties by location
 * @param location City, state, or ZIP code
 * @param status Listing status (for_sale, for_rent, sold)
 * @param page Page number for pagination
 * @return List of SearchCard objects
 */
@GetMapping("/search")
public List<SearchCard> searchProperties(
    @RequestParam String location,
    @RequestParam(required = false) String status,
    @RequestParam(defaultValue = "1") Integer page
);

/**
 * Get enriched property details by ZPID
 * @param zpid Zillow Property ID
 * @return EnrichedProperty with full details
 */
@GetMapping("/{zpid}")
public EnrichedProperty getPropertyDetails(@PathVariable String zpid);

```

Cashflow Controller (/api/analysis)

```

/**
 * Perform cashflow analysis on a property investment
 * @param request CashflowRequest containing all input parameters
 * @return CashflowResponse with summary and projections
 */
@PostMapping("/cashflow")
public CashflowResponse analyzeCashflow(@RequestBody CashflowRequest request);

```

Geo Controller (/api/geo)

```

/**
 * Text-based place search
 * @param text Search keyword
 * @param place Place name
 * @param city City name
 * @param state State code
 * @param country Country name
 * @param postcode ZIP/postal code
 * @return JSON response from Google API
 */
@GetMapping("/text")
public JsonNode textSearch(
    @RequestParam(required = false) String text,
    @RequestParam(required = false) String place,
    @RequestParam(required = false) String city,
    @RequestParam(required = false) String state,
    @RequestParam(required = false) String country,
    @RequestParam(required = false) String postcode
);

/**
 * Radius-based search around a point
 * @param lat Latitude of center point
 * @param lon Longitude of center point
 * @param radius Search radius in meters
 * @param text Optional search keyword
 * @return JSON response from Google API
 */
@GetMapping("/circle")
public JsonNode circleSearch(
    @RequestParam double lat,
    @RequestParam double lon,
    @RequestParam(defaultValue = "1000") int radius,
    @RequestParam(required = false) String text
);

```

5. Core Components

5.1 Application Entry Point

```
/**
 * Main Spring Boot application class.
 * Configures component scanning across all packages.
 */
@SpringBootApplication
@ComponentScan(basePackages = {
    "com.example.map",
    "com.example.realestate",
    "com.example.analysis"
})
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

5.2 Zillow Service

The ZillowService handles all Zillow API interactions:

```
/**
 * Service for interacting with Zillow API via RapidAPI.
 * Provides methods for property search, details, and enrichment.
 */
@Service
public class ZillowService {

    @Value("${zillow.rapidapi.host}")
    private String host;

    @Value("${zillow.rapidapi.key}")
    private String apiKey;

    /**
     * Search properties by location
     * @param location City, state, or ZIP code
     * @param status Listing status filter
     * @param page Page number
     * @return List of SearchCard results
     */
    public List<SearchCard> searchProperties(String location, String status, Integer page);

    /**
     * Get full enriched property data
     * Aggregates data from multiple API endpoints:
     */
}
```

```

    * - Property details
    * - Tax history
    * - Zestimate
    * - Rent estimate
    * - Comparables
    * - Images
    *
    * @param zpid Zillow Property ID
    * @return EnrichedProperty with all available data
    */
    public EnrichedProperty getEnrichedProperty(String zpid);
}

```

5.3 Cashflow Service

The core business logic for investment analysis:

```

/**
 * Core calculator for investment cashflow analysis.
 * Implements Year-1 KPIs and multi-year projections.
 *
 * Features:
 * - NOI (Net Operating Income) calculation
 * - Debt service with optional interest-only periods
 * - Cap rate analysis
 * - Cash-on-cash returns
 * - IRR calculation using Newton-Raphson method
 * - Multi-year projections with growth rates
 */
public class CashflowService {

    /**
     * Main analysis method
     * @param request Input parameters for analysis
     * @return Complete analysis with summary and projections
     */
    public CashflowResponse analyze(CashflowRequest request);
}

```

6. Data Models

6.1 CashflowRequest (Input DTO)

```

/**
 * Input DTO for investment cashflow analysis.
 * All monetary values in same currency (USD).
 * Percentage fields as decimals (e.g., 0.05 for 5%).
 */

```



```

public class CashflowRequest {

    // === IDENTIFICATION ===
    public String address;           // Property address
    public String city;              // City
    public String state;             // State code
    public String zip;               // ZIP code

    // === VALUATION ===
    public Double fmv;               // Fair Market Value
    public Double offerPrice;        // Purchase price
    public Double annualAppreciation; // Annual appreciation rate (0.03 = 3%)

    // === INCOME (Annual) ===
    public Double grossRentsAnnual;  // Total annual rent
    public Integer numberOfUnits;    // Number of units
    public Double parkingAnnual;     // Parking income
    public Double storageAnnual;     // Storage income
    public Double laundryVendingAnnual; // Laundry/vending income
    public Double otherIncomeAnnual; // Other income

    // === VACANCY & MANAGEMENT (Rates) ===
    public Double vacancyRate;       // Vacancy rate (0.05 = 5%)
    public Double managementRate;    // Management fee rate
    public Double repairsRate;       // Repairs rate (applied to gross rent)

    // === OPERATING EXPENSES (Annual flat amounts) ===
    public Double propertyTaxes;     // Annual property tax
    public Double insurance;         // Annual insurance
    public Double electricity;       // Annual electricity
    public Double gas;               // Annual gas
    public Double waterSewer;        // Annual water/sewer
    public Double cable;             // Annual cable/internet
    public Double caretaking;        // Caretaking costs
    public Double advertising;       // Advertising costs
    public Double associationFees;    // HOA fees
    public Double pest;              // Pest control
    public Double security;          // Security costs
    public Double trash;             // Trash removal
    public Double misc;              // Miscellaneous
    public Double commonAreaMaintenance; // CAM fees
    public Double capitalImprovements; // CapEx reserve
    public Double accounting;        // Accounting fees
    public Double legal;             // Legal fees
    public Double badDebts;          // Bad debt allowance
    public Double evictions;         // Eviction costs
    public Double otherExpenses;     // Other expenses
}

```

```

// === FINANCING - Primary Loan ===
public Double firstPrincipal;          // Loan amount
public Double firstRateAnnual;         // APR (0.07 = 7%)
public Integer firstAmortYears;        // Amortization term
public Integer firstInterestOnlyYears; // Interest-only years

// === FINANCING - Secondary Loan ===
public Double secondPrincipal;         // 2nd loan amount
public Double secondRateAnnual;        // 2nd loan APR
public Integer secondAmortYears;       // 2nd loan term

// === OTHER FINANCING ===
public Double otherMonthlyFinancingCosts; // Other monthly costs

// === CLOSING COSTS ===
public Double repairs;                 // Initial repairs
public Double repairsContingency;     // Repair contingency
public Double lenderFee;               // Lender fees
public Double brokerFee;              // Broker fees
public Double environmentals;          // Environmental reports
public Double inspections;             // Inspection costs
public Double appraisals;             // Appraisal costs
public Double transferTax;             // Transfer tax
public Double legalClose;             // Legal (closing)
public Double otherClosingCosts;       // Other closing costs

// === PROJECTION SETTINGS ===
public Integer holdYears;              // Holding period
public Double rentGrowth;              // Annual rent growth
public Double expenseGrowth;           // Annual expense growth
public Double exitCostRate;            // Selling cost rate

// === OPTIONS ===
public String managementBase;          // "EGI" or "GROSS_RENTS"
}

```

6.2 CashflowResponse (Output DTO)

```

/**
 * Output DTO for cashflow analysis results.
 */
public class CashflowResponse {

    public Summary summary;          // Key metrics
    public List<YearRow> projection; // Year-by-year data

    /**
     * Summary metrics for the investment

```

```

*/
public static class Summary {
    // === PRICING ===
    public Double rpp;           // Real Purchase Price
    public Double cashToClose;   // Cash required at closing

    // === YEAR-1 INCOME/EXPENSE ===
    public Double totalIncomeY1; // Total income
    public Double vacancyLossY1; // Vacancy loss
    public Double egiY1;         // Effective Gross Income
    public Double totalExpensesY1; // Total expenses
    public Double noiY1;         // Net Operating Income

    // === DEBT METRICS ===
    public Double annualDebtServiceY1; // Annual debt payment
    public Double dscrY1;              // Debt Service Coverage Ratio

    // === PROFITABILITY ===
    public Double capRatePPY1;         // Cap rate (Purchase Price)
    public Double capRateFMVY1;        // Cap rate (FMV)
    public Double grmY1;               // Gross Rent Multiplier
    public Double avgRentPerUnitY1;    // Average rent per unit
    public Double monthlyProfitY1;     // Monthly profit
    public Double cashflowPerUnitPerMonthY1; // CF per unit/month

    // === LEVERAGE ===
    public Double ltvFMV;              // Loan-to-Value (FMV)
    public Double ltppPP;              // Loan-to-Price (PP)

    // === RETURNS ===
    public Double cashOnCashY1;        // Cash-on-Cash Return
    public Double equityROIY1;         // Equity ROI
    public Double appreciationROIY1;   // Appreciation ROI
    public Double totalROIY1;          // Total ROI
    public Double forcedAppreciationROIY1; // Forced appreciation

    // === MULTI-YEAR ===
    public Double irr;                 // Internal Rate of Return
    public Double equityMultiple;      // Equity Multiple
    public Double saleProceedsNet;     // Net proceeds at exit
}

/**
 * Yearly projection data
 */
public static class YearRow {
    public int year;                 // Year number
    public Double totalIncome;       // Total income

```

```

        public Double vacancyLoss;           // Vacancy loss
        public Double egi;                   // Effective Gross Income
        public Double management;            // Management expense
        public Double repairsRateBased;      // Repairs expense
        public Double totalExpenses;         // Total expenses
        public Double noi;                   // Net Operating Income
        public Double debtService;           // Debt service
        public Double cashFlowBeforeTax;     // Cash flow before tax
        public Double endingBalanceFirst;    // 1st loan balance
        public Double endingBalanceSecond;   // 2nd loan balance
        public Double propertyValue;         // Property value
    }
}

```

6.3 EnrichedProperty

```

/**
 * Comprehensive property data model.
 * Aggregates data from multiple Zillow API endpoints.
 */
public class EnrichedProperty {

    // === IDENTIFICATION ===
    public String zpid;           // Zillow Property ID
    public String address;        // Full address
    public String status;         // Listing status
    public String propertyType;   // Property type

    // === BASIC INFO ===
    public Double price;          // List price
    public Double bedrooms;       // Number of bedrooms
    public Double bathrooms;      // Number of bathrooms
    public Double livingArea;     // Living area (sqft)
    public Double lotAreaValue;   // Lot size
    public Integer yearBuilt;     // Year built

    // === LOCATION ===
    public Double lat;            // Latitude
    public Double lon;           // Longitude

    // === ESTIMATES ===
    public Double zestimate;      // Zillow estimate
    public Double rentEstimate;   // Rent Zestimate

    // === COSTS ===
    public Double hoaMonthly;     // Monthly HOA
    public Double annualTax;      // Annual property tax
}

```

```

// === MEDIA ===
public String imgSrc;           // Primary image URL
public List<String> photoUrls;  // All photo URLs
public List<String> tour3dUrls; // 3D tour URLs
public List<String> floorPlanUrls; // Floor plan URLs

// === FEATURES ===
public List<String> heating;    // Heating types
public List<String> cooling;     // Cooling types
public List<String> parkingFeatures; // Parking features

// === COMPARABLES ===
public Integer compsCount;      // Number of comps
public Double compsMedianSoldPrice; // Median comp price
public Double compsMedianPpsf;   // Median price per sqft

// === DERIVED ===
public Double pricePerSqft;      // Price per sqft
public Double lotToBuildingRatio; // Lot to building ratio

// === LINKS ===
public String zillowWebUrl;      // Zillow listing URL
}

```

7. Business Logic

7.1 Cashflow Calculation Algorithm

```

/**
 * STEP 1: Calculate Year-1 Income Stack
 */
double totalIncomeY1 = grossRents + parking + storage + laundry + otherIncome;
double vacancyLossY1 = totalIncomeY1 * vacancyRate * -1;
double egiY1 = totalIncomeY1 + vacancyLossY1; // Effective Gross Income

/**
 * STEP 2: Calculate Year-1 Expenses
 */
double managementY1 = managementRate * managementBase; // Based on EGI or Gross Rents
double repairsY1 = repairsRate * grossRents;
double totalExpensesY1 = managementY1 + repairsY1 + sumOfAllOtherExpenses;
double noiY1 = egiY1 - totalExpensesY1; // Net Operating Income

/**
 * STEP 3: Calculate Debt Service
 */
// Using amortization formula:  $M = P * [r(1+r)^n] / [(1+r)^n - 1]$ 

```

```

double annualDebtService = firstLoanPayment + secondLoanPayment + otherFinancing;
double dscrY1 = noiY1 / annualDebtService; // Debt Service Coverage Ratio

/**
 * STEP 4: Calculate Returns
 */
double rpp = offerPrice + allClosingCosts; // Real Purchase Price
double cashToClose = rpp - firstPrincipal - secondPrincipal;
double capRateY1 = noiY1 / offerPrice;
double cashOnCashY1 = (noiY1 - annualDebtService) / cashToClose;

/**
 * STEP 5: Multi-Year Projection
 */
for (int year = 1; year <= holdYears; year++) {
    // Grow income by rentGrowth rate
    // Grow expenses by expenseGrowth rate
    // Calculate debt service (accounting for interest-only periods)
    // Track loan balances
    // Track property appreciation
}

/**
 * STEP 6: Calculate Exit Metrics
 */
double salePrice = offerPrice * pow(1 + appreciation, holdYears);
double saleCosts = salePrice * exitCostRate;
double netSaleProceeds = salePrice - saleCosts - remainingLoanBalance;
double irr = calculateIRR(cashflows); // Newton-Raphson method
double equityMultiple = totalDistributions / initialInvestment;

```

7.2 IRR Calculation (Newton-Raphson)

```

/**
 * Calculate Internal Rate of Return using Newton-Raphson iteration.
 *
 * NPV formula:  $(Cf_t / (1+r)^t) = 0$ 
 *
 * @param cashflows List of cash flows (index 0 = initial investment, negative)
 * @param guess Initial guess for IRR
 * @return IRR as decimal (null if not converged)
 */
private static Double irr(List<Double> cashflows, double guess) {
    double x = guess;
    for (int i = 0; i < 50; i++) { // Max 50 iterations
        double f = 0, df = 0;
        for (int t = 0; t < cashflows.size(); t++) {
            double ct = cashflows.get(t);

```

```

        double d = Math.pow(1 + x, t);
        f += ct / d; // NPV
        if (t > 0) df += -t * ct / Math.pow(1 + x, t + 1); // Derivative
    }
    double x1 = x - f / df; // Newton-Raphson step
    if (Math.abs(x1 - x) < 1e-7) return x1; // Converged
    x = x1;
}
return null; // Did not converge
}

```

8. Testing

Running Tests

```

# Run all tests
mvn test

# Run specific test class
mvn test -Dtest=CashflowServiceTest

# Run with coverage report
mvn test jacoco:report

```

Test Structure

```

src/test/java/com/example/
    € analysis/
    ?     € service/
    ?         € CashflowServiceTest.java    # Unit tests for cashflow
    € realestate/
    ?     € service/
    ?         € ZillowServiceTest.java      # Integration tests for Zillow
    € map/
    € geo/
    € GeoControllerTest.java                # API endpoint tests

```

Writing Tests

```

/**
 * Example test for CashflowService
 */
@Test
public void testBasicCashflowCalculation() {
    CashflowRequest request = new CashflowRequest();
    request.offerPrice = 500000.0;
    request.grossRentsAnnual = 48000.0;
    request.vacancyRate = 0.05;
}

```

```

request.propertyTaxes = 6000.0;
request.insurance = 2400.0;
request.firstPrincipal = 400000.0;
request.firstRateAnnual = 0.07;
request.firstAmortYears = 30;
request.holdYears = 10;

CashflowService service = new CashflowService();
CashflowResponse response = service.analyze(request);

assertNotNull(response.summary);
assertTrue(response.summary.noiY1 > 0);
assertTrue(response.summary.dscrY1 > 1.0);
assertEquals(10, response.projection.size());
}

```

9. Code Style Guidelines

Java Style

```

/**
 * Class-level Javadoc explaining purpose
 */
public class ExampleService {

    // Constants at top
    private static final int MAX_RETRIES = 3;

    // Instance variables
    private final HttpClient httpClient;

    /**
     * Constructor with dependency injection
     */
    public ExampleService(HttpClient httpClient) {
        this.httpClient = httpClient;
    }

    /**
     * Method-level Javadoc for public methods
     * @param param Description of parameter
     * @return Description of return value
     * @throws ExceptionType When this exception is thrown
     */
    public Result doSomething(String param) throws SomeException {
        // Implementation
    }
}

```



```

    // Private helper methods at bottom
    private void helperMethod() {
        // ...
    }
}

```

Formatting Rules

- **Indentation:** 2 spaces (no tabs)
 - **Line length:** Max 120 characters
 - **Braces:** K&R style (opening brace on same line)
 - **Imports:** Organized, no wildcards
-

10. Contributing

Git Workflow

```

# 1. Create feature branch
git checkout -b feature/your-feature-name

# 2. Make changes and commit
git add .
git commit -m "feat: add new feature description"

# 3. Push and create PR
git push origin feature/your-feature-name

```

Commit Message Format

<type>: <description>

[optional body]

[optional footer]

Types: feat, fix, docs, style, refactor, test, chore

11. Generating Documentation

Using Doxygen (Optional)

1. Install Doxygen:

```

# Windows (Chocolatey)
choco install doxygen.install

# macOS

```

```
brew install doxygen
```

```
# Linux
```

```
sudo apt install doxygen
```

2. Create Doxyfile:

```
doxygen -g
```

3. Configure Doxyfile:

```
PROJECT_NAME = "Real Estate Backend"
INPUT = src/main/java
RECURSIVE = YES
EXTRACT_ALL = YES
GENERATE_HTML = YES
GENERATE_LATEX = NO
JAVADOC_AUTOBRIEF = YES
```

4. Generate documentation:

```
doxygen Doxyfile
```

Using Pandoc for PDF Export

1. Install Pandoc:

```
# Windows
```

```
choco install pandoc
```

```
# macOS
```

```
brew install pandoc
```

```
# Linux
```

```
sudo apt install pandoc texlive-latex-base
```

2. Export to PDF:

```
pandoc README.md USER_GUIDE.md DEVELOPER_GUIDE.md -o documentation.pdf \
  --toc \
  --toc-depth=3 \
  -V geometry:margin=1in \
  -V fontsize=11pt
```

Document History

Version	Date	Author	Changes
1.0.0	Dec 2025	Team	Initial release